



1

15

1800 sp

نظري

كلية الهندسة المعلوماتية

السنة الخامسة - هندسة البرمجيات



Introduction & Design Pattern

د. باسم قصيبة

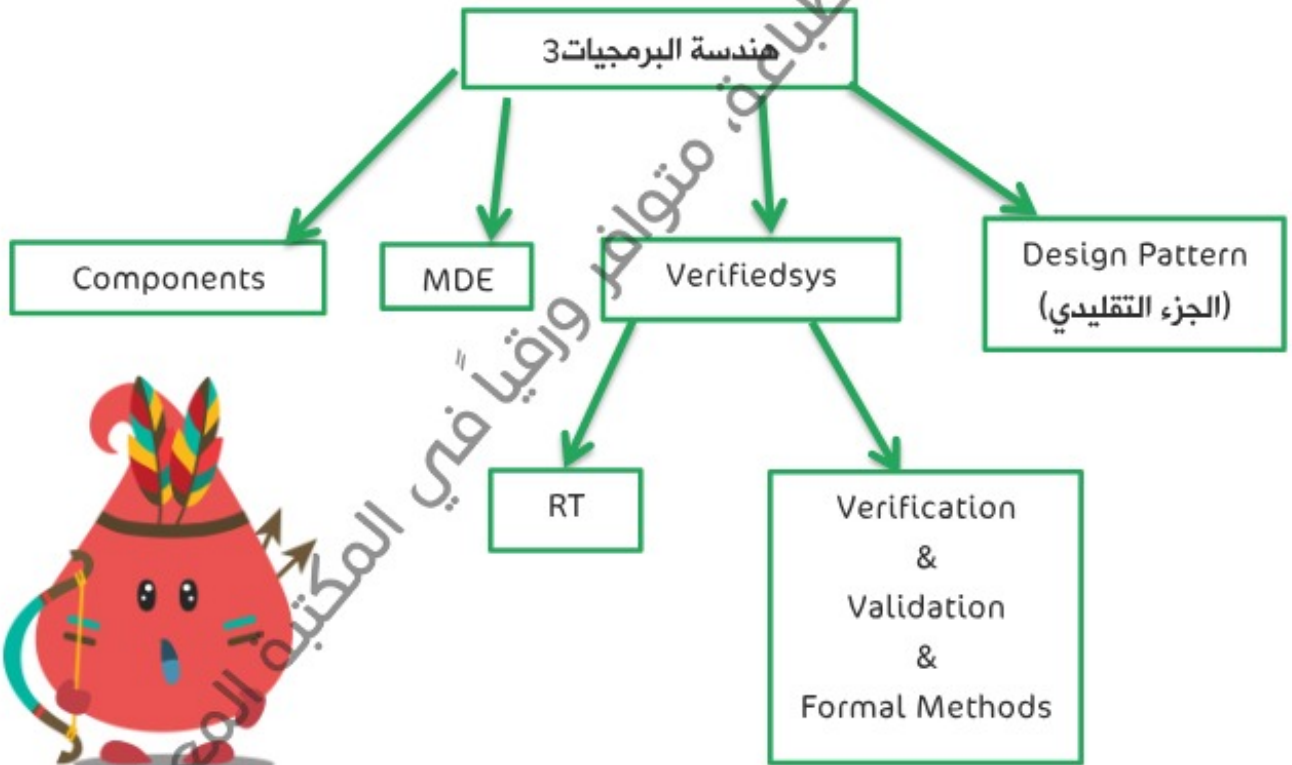
7/10/2024

هندسة برمجيات ٣

RB Informatics;

السلام عليكم ورحمة الله وبركاته

اهلا بكم في مقرر هندسة البرمجيات 3 والذي سنتعرف فيه على عدة مفاهيم منها:



أولاً: ما الفرق بين Validation و Verification:

كما نعلم في الهندسة 1 بأن:

Validation: هي التأكد من أن المتطلبات تلبي احتياج الزبون (are we build the right product ?).

Verification: هي التأكد من أن المتطلبات أنجزت بالشكل الصحيح (are we build the product right ?).

هذا بالمعنى النظري ، أما بالمعنى العملي:

Validation

نقوم أولاً بجمع المتطلبات ويتم تمثيلها بعدد من المخططات بعدها يتم بناء النظام بناءً على المخططات بعد ذلك يتم اختبار هذا النظام بناءً على المتطلبات.

Requirements → Diagrams → System → Validation

نستنتج أن Validation هو اختبار النظام بناءً على المتطلبات.

■ مثال على Validation:

قمنا بجمع المتطلبات لبناء تطبيق يمكن فيه:

1. فتح صورة. 2. البحث عن صورة. 3. تعديل صورة.

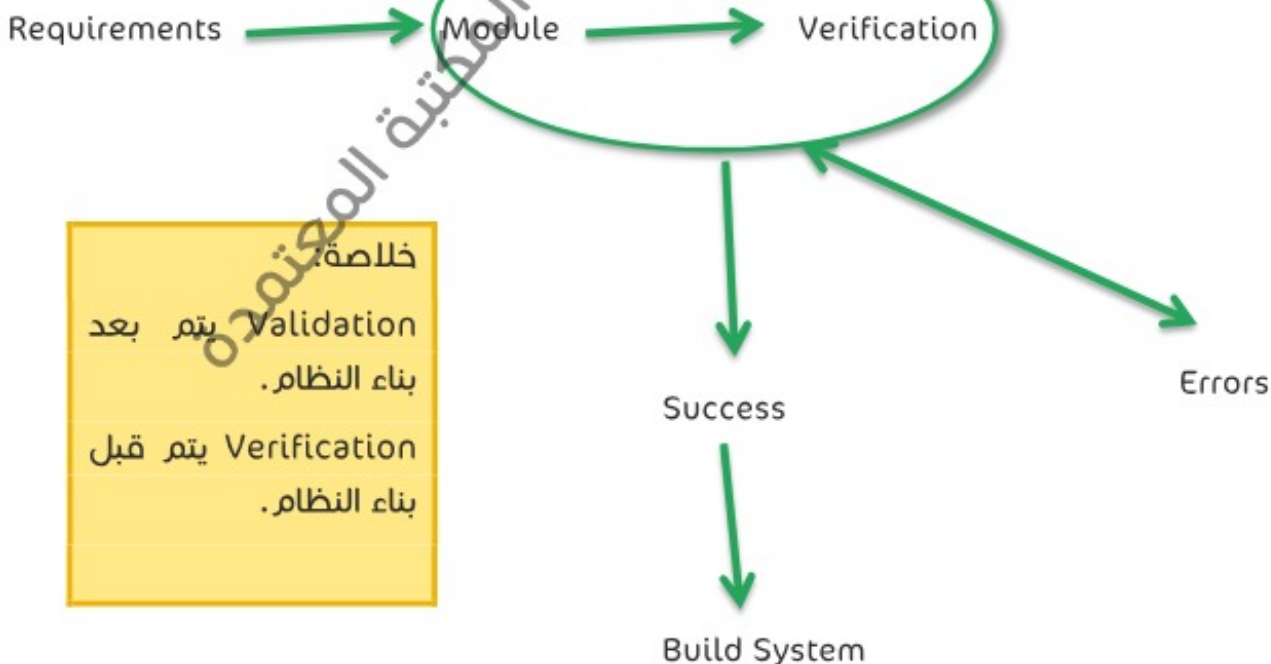
نقوم ببناء المتطلبات وبناء مخططات لها ومن ثم بناء النظام واختباره.

والاختبار (Validation) يكون على سبيل المثال:

- عند البحث عن صورة يكون الناتج مطابق للصورة المراد البحث عنها.
- عند تعديل صورة هل يتم إجراء التعديلات عليها بشكل صحيح أم تحدث أخطاء.

Verification

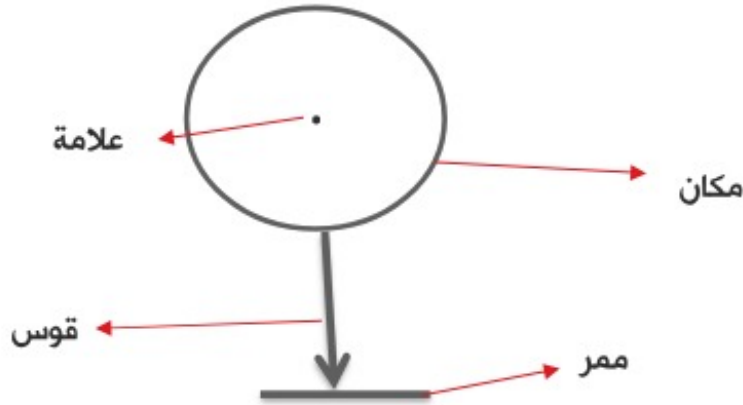
يتم بناء نموذج للنظام المراد برمجته ، حيث يتم تطبيق اختبارات معينة على هذا النموذج ، فإن نجحت هذه الاختبارات أقوم ببناء النظام ، و إن لم تنجح هذه الاختبارات أقوم بالتعديل على هذا النموذج حتى تصبح هذه الاختبارات ناجحة ثم أقوم ببناء النظام.



شبكات بتري 1962

تستخدم في اختبار الأنظمة المعقدة التي تحوي على Threads أو الأنظمة المتوازية (الأنظمة التي تعمل على التفرع).

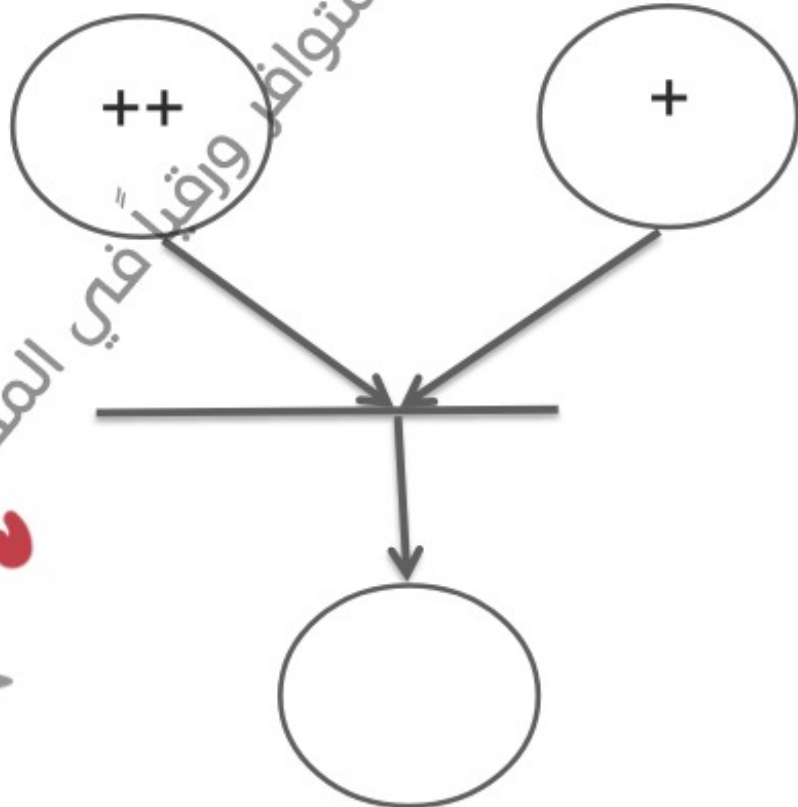
مفاهيم شبكات بتري



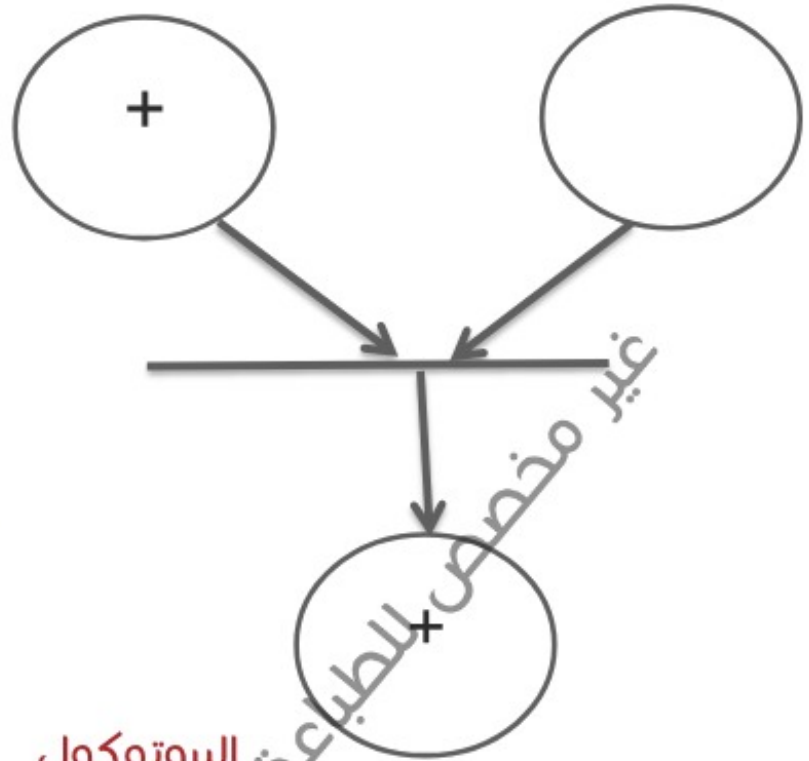
■ الممر قابل للعبور إذا كان كل مكان دخل له يحوي علامة واحدة على الأقل.

■ نغير الممر من خلال حذف علامة بكل مكان دخل و إضافة علامة بكل مكان خرج لهذا الممر.

مثال: حدد فيما إذا كان الممر التالي قابل للعبور أم لا:



هذا الممر يمكن عبوره لأن نقطة الدخول في المكان الأول يقابلها نقطة واحدة على الأقل في المكان الثاني للدخول وبالتالي:



البروتوكول

هو مجموعة من القواعد ، على سبيل المثال:

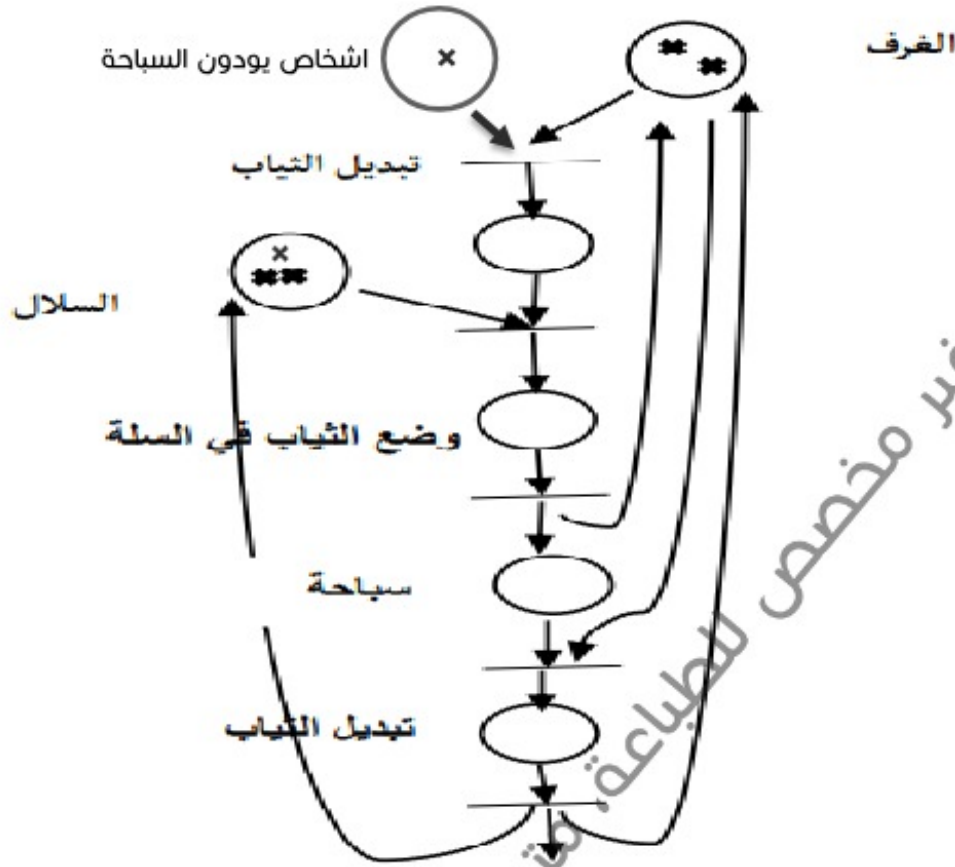
بروتوكول إدارة مسبح:

1. يدخل الشخص
2. يحصل على غرفة تبديل
3. يحصل على سلة
4. يُبدل ثيابه
5. يضع الثياب في السلة
6. يحرر الغرفة
7. يسبح
8. يحصل على غرفة تبديل
9. يغير ثيابه
10. يحرر السلة والغرفة
11. يخرج



السؤال: هل يوجد deadlock في هذا البروتوكول؟

لنقم برسم شبكة بتري لهذا البروتوكول

**الحل:**

كما في المثال السابق (صفحة 3) نأخذ علامة من كل أماكن الدخول ونضيف علامة بكل أماكن الخروج من أجل كل ممر عبور.

نقوم بالتحقق المرئي visual أي أن نناقش سيناريو الخطأ الممكن حدوثه ، وهناك تحقق رياضي.

التحقق ال visual :

بفرض لدينا غرفتين و 3 سلال :

السيناريو الطبيعي:

يدخل أول شخص يحصل على غرفة تبادل ويقوم بعبور الممر الأول ويقوم بتبادل الثياب ويأخذ سلة يضع الثياب فيها ويقوم بعبور الممر الثاني ومن ثم يحرر الغرفة ، ثم يذهب للسباحة ثم يحصل على غرفة من جديد ثم يقوم بتبادل الثياب ومن ثم يعيد (يحرر) كل من الغرفة والسلة.

حالة الاختناق Deadlock:

1. يدخل الشخص الأول يحصل على غرفة -> يبدل الثياب -> يأخذ سلة -> يضع الثياب في السلة -> يحرر الغرفة -> يذهب للسباحة.
2. يدخل الشخص الثاني يحصل على غرفة -> يبدل الثياب -> يأخذ سلة -> يضع الثياب في السلة -> يحرر الغرفة -> يذهب للسباحة.

3. يدخل الشخص الثالث (إمكانية الحصول على غرفة) يحصل على غرفة - > يبدل الثياب - > يأخذ سلة - > يضع الثياب في السلة - > يحرر الغرفة - > يذهب للسباحة.
4. يدخل الشخص الرابع (إمكانية الحصول على غرفة) يحصل على غرفة - > يبدل الثياب - > (عدم إمكانية عبور الممر الثاني لعدم توافر السلالم) - > فتبقى الغرفة محجوزة من قبل هذا الشخص.
5. يدخل الشخص الخامس (إمكانية الحصول على غرفة) يحصل على غرفة - > يبدل الثياب - > (عدم إمكانية عبور الممر الثاني لعدم توافر السلالم) - > فتبقى الغرفة محجوزة من قبل هذا الشخص.

حالة الـ Deadlock: عدم إمكانية دخول الأشخاص والحصول على غرف وعدم إمكانية خروج الأشخاص من المسبح لعدم توافر غرف.

ملاحظات:

شبكات بتري تقوم بإعطاء كامل السيناريوهات التي يمكن أن تمر بها المسألة (بما فيها حالات الاختناق Deadlock).
في شبكات بتري نقول عن سيناريو أنه Deadlock عندما يكون ولا ممر فيه قابل للعبور.

مميزات شبكات بتري:

1. المحاكاة (من خلال تمثيل عدة سيناريوهات للنظام المدروس).
2. شجرة التغطية (حيث تغطي شبكات بتري جميع الحالات التي يمر بها النظام المدروس).

أخطاء يجب تجنبها عند رسم شبكات بتري:

1. من الخطأ وضع قوس بين ممرين

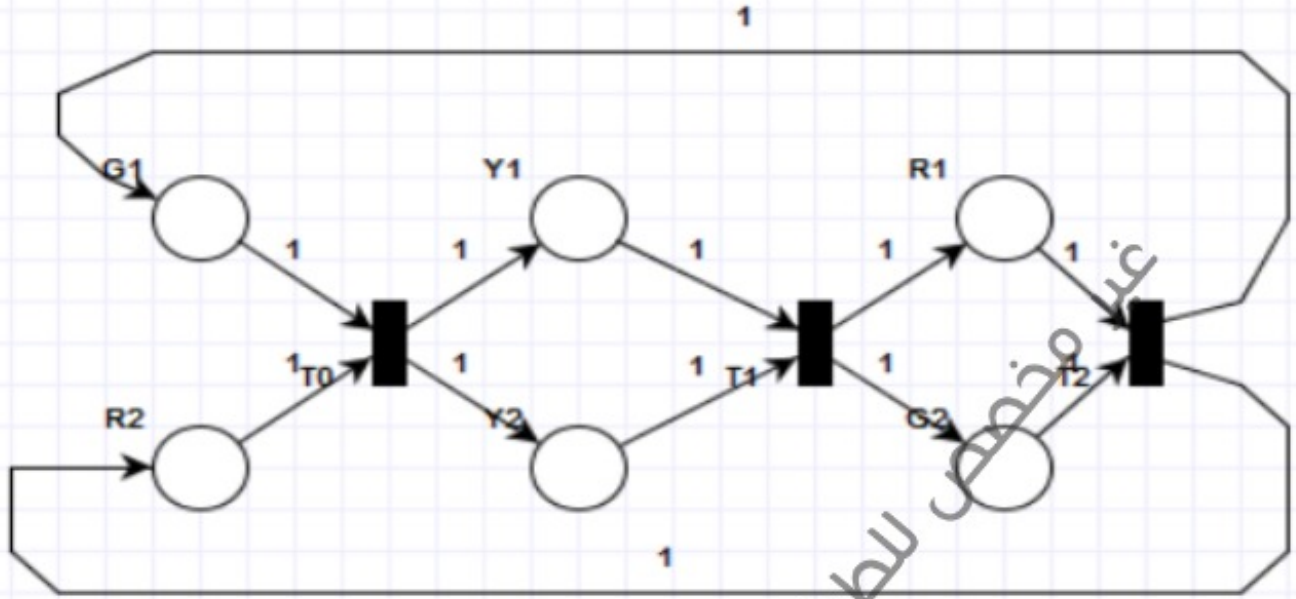
خطأ



2. من الخطأ وضع قوس بين مكانين

خطأ





هناك حالتين مرفوضتين يجب تجنبها في نظام إشارة المرور مثل:

1. الإشارة الأولى والثانية لونها أخضر (لأن ذلك سيؤدي إلى حوادث مرورية).
2. الإشارة الأولى والثانية لونها أحمر (سيستوقف المرور ويضيع الزمن بلا فائدة).

كلتا الحالتين السابقتين مرفوض ولتجنبها نقوم بتمثيل نظام إشارة المرور عن طريق شجرة التغطية:

نُمثل شجرة التغطية بشعاع كالتالي:

$$\begin{array}{c}
 R1 \\
 G1 \\
 Y1 \\
 R2 \\
 G2 \\
 Y2
 \end{array}
 \begin{bmatrix}
 1 \\
 0 \\
 0 \\
 0 \\
 1 \\
 0
 \end{bmatrix}
 \longrightarrow
 \begin{bmatrix}
 0 \\
 1 \\
 0 \\
 0 \\
 1 \\
 0
 \end{bmatrix}$$

الحالة البدائية الحالة التالية

كما نرى الحالة البدائية مقبولة لأن $G1 \neq G2$ و $R1 \neq R2$ أما الحالة التالية فيجب تجنبها لأن $G1 = G2$ و $R1 = R2$.

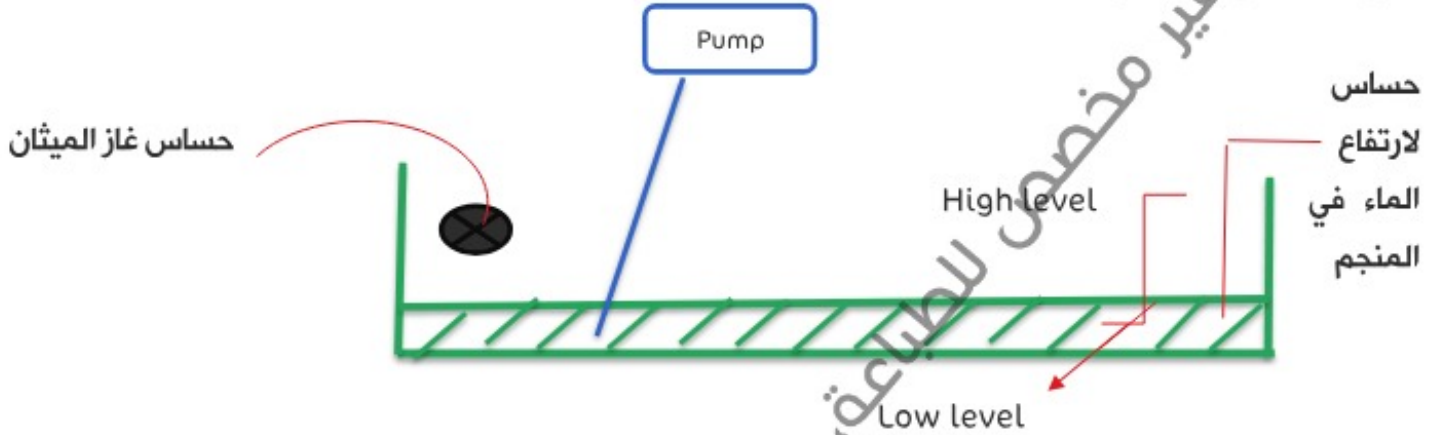
سنقوم بالتعمق أكثر في شبكات بتري وكيفية رسمها وشجرة التغطية في محاضرات قادمة.

RT (real time system)

هو نظام حيث أن صحته لا تعتمد فقط على صحة نتائج حساباته المنطقية و إنما على الزمن الذي تم إنجاز العمليات فيها.

أي أن RT هي نظام معتمد على الزمن ، ومهامه تتم ضمن فترة زمنية معينة حسب الزمن الداخلي للنظام.

مثال: نظام الإنذار ضمن منجم فحم



في هذا النظام تعمل المضخة عند الوصول إلى High level وتتوقف عند الوصول إلى مستوى الماء حتى low level.

كما أن المضخة يجب ألا تعمل إذا كانت نسبة الميثان أعلى من حد حرج و إلا (شرارة مضخة المياه مع الميثان ستؤدي إلى انفجار).

في هذا النظام لدينا مهمتين:

1. قياس غاز الميثان.
2. قياس مستوى الماء في المنجم.

<- لنفرض أن المهمتين دوريتين ، نلاحظ أن دور مهمة قياس الميثان صغير T1 وذلك لأنه خطير وهو سبب لانفجار فخلال فترة صغيرة يمكن أن يتدفق غاز الميثان ويملأ الجو بنسبة معينة وبالتالي عند وجود أي شرارة يحدث الانفجار.

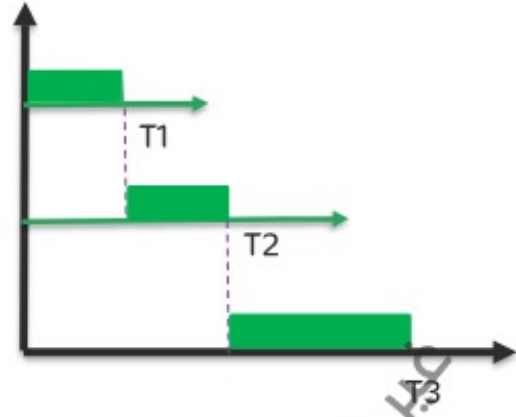
يتم حساب T1 الأسوأ أي ماهي أكبر غزارة محتملة لتدفق غاز الميثان وبالتالي يجب اختيار T1 بحيث إذا تدفق غاز الميثان بأكبر غزارة محتملة فإنه يجب أن يتحسس لذلك ونأخذ الإجراءات المناسبة لإيقاف المضخة و آلات الحفر.

لدينا أيضاً مهمة دورية لقياس مستوى الماء لكنها ليست حرجية كمهمة قياس الميثان ، حيث أن الماء لن يتدفق بالسرعة الكبيرة ليشكل خطر على حياة الناس.

وبالتالي فإن دوره T2 وهذا الدور كافٍ لتحسس مستوى الماء قبل أن يصبح مهدداً لحياة الناس.

المخطط السابق يوضح ال real time للمنجم بحيث نلاحظ أن الدور الخاص لقياس الميثان T1 حرج و يجب قياسه في وقت قصير ، أما الدور الخاص لقياس الماء T2 يستغرق قياسه وقت أكثر لأنه أقل خطورة.

ملاحظة: لو أن T2 سيتم الوصول له دون المرور ب T1 عندها لن يكون النظام real time



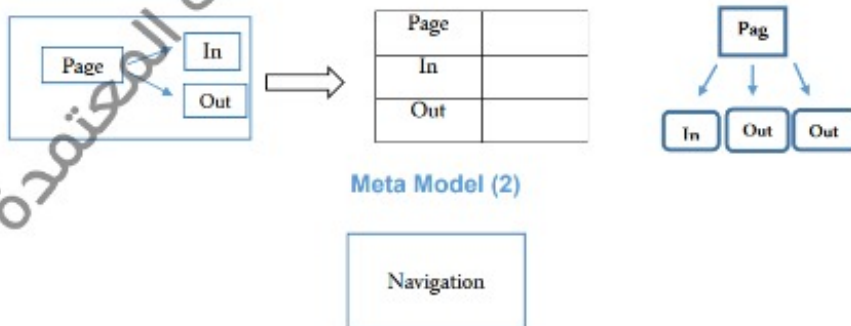
MDE (Model Driven Engineering)

الهندسة المقادة بالنماذج تُستخدم للتعبير عن الاستخدام النظامي للبنى البرمجية أو النماذج كحقائق أو عناصر أولية خلال عملية تصميم البرمجيات.

فهو منهج لمطوري البرمجيات يؤكد على ضرورة استعمال النماذج في عملية التحديد ، التطوير ، التحليل ، والتحقق و إدارة الأنظمة البرمجية وبالتالي لم يعد التطوير باستخدام الشيفرة البرمجية (code) و إنما باستخدام النماذج (Model) ، ولكن النماذج غرضية التوجه غير فعالة دائماً.

مثال: عند تصميم تطبيقات ويب باستخدام ال PHP (باعتبار php لغة غير مُهيكله) ففي حال قمنا بتصميم صفحة وتحتاج إلى تعديل ، فيجب معرفة هذه الصفحة و علاقتها بالصفحات الأخرى فنحتاج إلى قراءة الصفحة كاملة.

فإذا أردنا تحليل نظام صفحة الويب المعتمد هو استخدام لغة النمذجة UML لكن الأفضل استخدام Meta Model ، حيث نقوم بنمذجة مفردات المسألة مثلا الصفحة (page) هي مجموعة من المدخلات (in) و المخرجات (out).



وممكن ربط النموذجين السابقين وتشكيل نموذج جديد.

سابقا لم يكن هناك فصل للأدوار بين المستخدمين أي أن الشخص المصمم للصفحة هو المسؤول عن برمجة ال control للصفحات ، في حال تمكنا باستخدام ال meta Model من أيجاد أداة برمجية تقوم بتجميع

الصفحات وتقوم ببرمجة ال control و أداة ثانية من أجل تصميم الصفحات وبالتالي فمننا بتوزيع الادوار بين المستخدمين. وبالتالي عملية مهمة بالهندسة .

ملاحظة : تكمن الاستفادة من نموذج ال MDE في تطوير المنتجات البرمجية وهندسة فكرة التطوير.

مثال : نحتاج لوقت طويل للقيام بتأهيل عامل ليقوم بتصنيع تلفاز بشكل كامل، أما إذا تم تصنيع التلفاز على أجزاء فإن كل فرد من العمال يقوم بمهمة محددة فدرجة التأهيل الخاصة بالعامل الواحد ستكون صغيرة ، ولكن بتجميع أعمالهم سوياً ، هي التي تصنع التلفاز بشكل كامل وهكذا يتم توفير الجهد والوقت.

- ال Process هو مجموعة من الخطوات المحددة للوصول إلى هدف محدد.
- الفائدة من ال Process هو القدرة على تحديد المهام بشكل واضح وتقسيماها (توزيعها) على الأشخاص.
- تكمن أهمية MDE بأنها طريقة بديلة عن UML حيث تستخدم MDE في تصميم أنظمة التشغيل وأجهزة التحكم.

Design Patterns: نماذج التصميم وهو فرع مهم من فروع علوم هندسة البرمجيات ، والذي يهدف إلى الاستخدام المتكرر لحلول وضعت لمشاكل تعتبر في حد ذاتها متكررة أو متواجدة بكثرة في عملية تصميم البرمجيات. لا تنطبق هذه النظرية على تصميم البرمجيات فحسب ، بل تتعداه إلى عدة علوم أخرى مثل هندسة المباني وهندسة الإلكترونيات وحتى الطبخ.

من الأخطاء المنتشرة اعتبار نماذج التصميم حلولاً كاملة أو جاهزة للاستخدام مباشرةً ، فهي لا تغدو أن تكون إلا نموذجاً كما يدل اسمها يحتاج للتكييف والتحديد أكثر لكي يواجه مشكلة محددة. أغلب نماذج التصميم تعتمد على التوجه الكائني في البرمجة. لذلك نجدها تضع تصوراً قائماً على التفاعلات والعلاقات الممكنة بين مختلف الفئات المكونة للبرنامج.

Design Patterns

يتضمن 23 نموذجاً تندرج تحت 3 أقسام:

- Creational: يتم التعامل مع عملية خلق الاغراض الجديدة وهي أكثر عملية زادت Dependence بالنظام.
- Structural: تجميع الأغراض.
- Behavioral: التحكم في سلوك النظام.

أعراض النظام السيء:

- ❖ Rigidity (القساوة) : تغير في مكان ما يؤدي إلى التغير بمكان اخر.
- ❖ Frailty (الهشاشة)

❖ Immobility (الجمود): صف يعتمد على صفوف اخرى فعند استخدامه بغير مكان سأضطرب لجلب ال Package بكامله.

❖ Viscosity (اللزوجة): إذا قمنا بتصميم نموذج لحل مشكلة ما و أردنا فيما بعد إجراء تعديل في بنية النظام فنكون أمام خيارين:

1. طريقة تحافظ على التصميم.
2. طريقة لا تحافظ على التصميم.

وإذا كانت الطريقة التي لا تحافظ على التصميم أسهل يكون التصميم لزوج، لأنه يجب أن يكون التصميم يراعي التغيرات التي يمكن أن تحدث على النظام.

Principles of Object Oriented Class Design

The Open Closed Principle (OCP):

«A module should be open for extension but closed for modification»

مفهوم ال (ocp) أن النظام قابل للتوسع فيمكن إضافة وظائف جديدة (open)، لكن لا يمكن إجراء أي تعديل في بنية الصفوف الموجودة مسبقاً (closed).

مثال: مسألة معالجة الصور:

اكتب برنامج لمعالجة الصور من أنواع مختلفة (GIF, BMP, JPG) حيث يسمح هذا البرنامج بمعرفة عرض و ارتفاع الصورة علماً بأن هذه المعالجة تعتمد على نوع الصورة.

ملاحظة: نريد في هذا البرنامج معالجة الصورة بأسلوب متماثل و إمكانية إضافة أنواع جديدة من الصور. الحل الأول:

```
public class Image
{
    private String type;
    public Image(String type) { this.type = type; }
    .....
    public int getWidth() {
        if (this.type.equals("jpg"))
            // procesing to determine the width of jpg image
        else if (this.type.equals("gif"))
            // procesing to determine the width of gif image
        else if (this.type.equals("bmp"))
            // procesing to determine the width of bmp image
        .....
    }
}

public class ImageProcessor
{
    public int getImageWidth(Image img) {return img.getWidth(); }
    public int getImageHeight(Image img) {return img.getHight(); }
}
```

وهو حل سيء لأنه يراعي الحالات المعروفة ضمن الكود فقط (GIF, BMP, JPG) فإذا أردنا معالجة حالة أخرى فإننا بحاجة إلى تعديل الكود وبالتالي مفهوم ال OCP غير محقق.

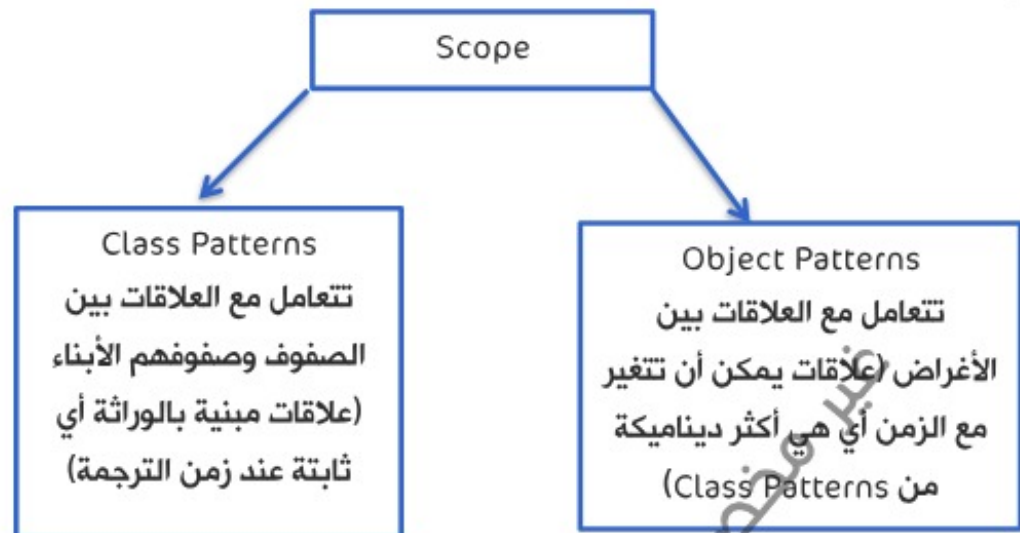
الحل الجيد:

```
public interface Image
{
    public int getWidth();
    ....
}
public class GifImage implements Image
{
    public int getWidth() { //processing to determine the width of gif image}
}
public class JpgImage implements Image
{
    public int getWidth() { //processing to determine the width of jpg image}
}
-----
public class ImageProcessor
{
    public int getImageWidth(Image img)
    {
        return img.getWidth();
    }
}
```

شرح: نقوم بإنشاء interface اسمه image نعرّف ضمنه إجرائية getWidth() ونعرّف كلاسين GifImage, JpgImage. في حال أضفنا نوع صورة جديدة فقط نحتاج لإضافة صف جديد ، وبذلك عملية التعديل تكون بإضافة صف جديد دون تعديل في بنية الكود السابقة ، وبالتالي تحققت مفهوم ال OCP.

Classification of Design Patterns

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Flyweight Observer State Strategy Visitor



Creational Patterns: تهتم بعملية إنشاء الأغراض.

Class Creational Patterns ✓ تؤجل إنشاء الأغراض إلى الصفوف الأبناء.

Object Creational Patterns ✓ توكل هذه العملية لغرض آخر.

Structural Patterns: تهتم بتجميع الصفوف و الأغراض لتشكيل بنى أوسع.

Class Creational Patterns ✓ تستخدم الوراثة لتجميع الصفوف.

Object Creational Patterns ✓ تستخدم composition لتجميع الأغراض.

Behavioral Patterns: توصف طرق تفاعل الصفوف و الأغراض وتوزيع المسؤولية فيما بينهم.

Class Creational Patterns ✓ تستخدم الوراثة لتوزيع المسؤولية.

Object Creational Patterns ✓ تجمع الأغراض وتصف كيفية تعاونهم لإنجاز مهمة.

Visitor

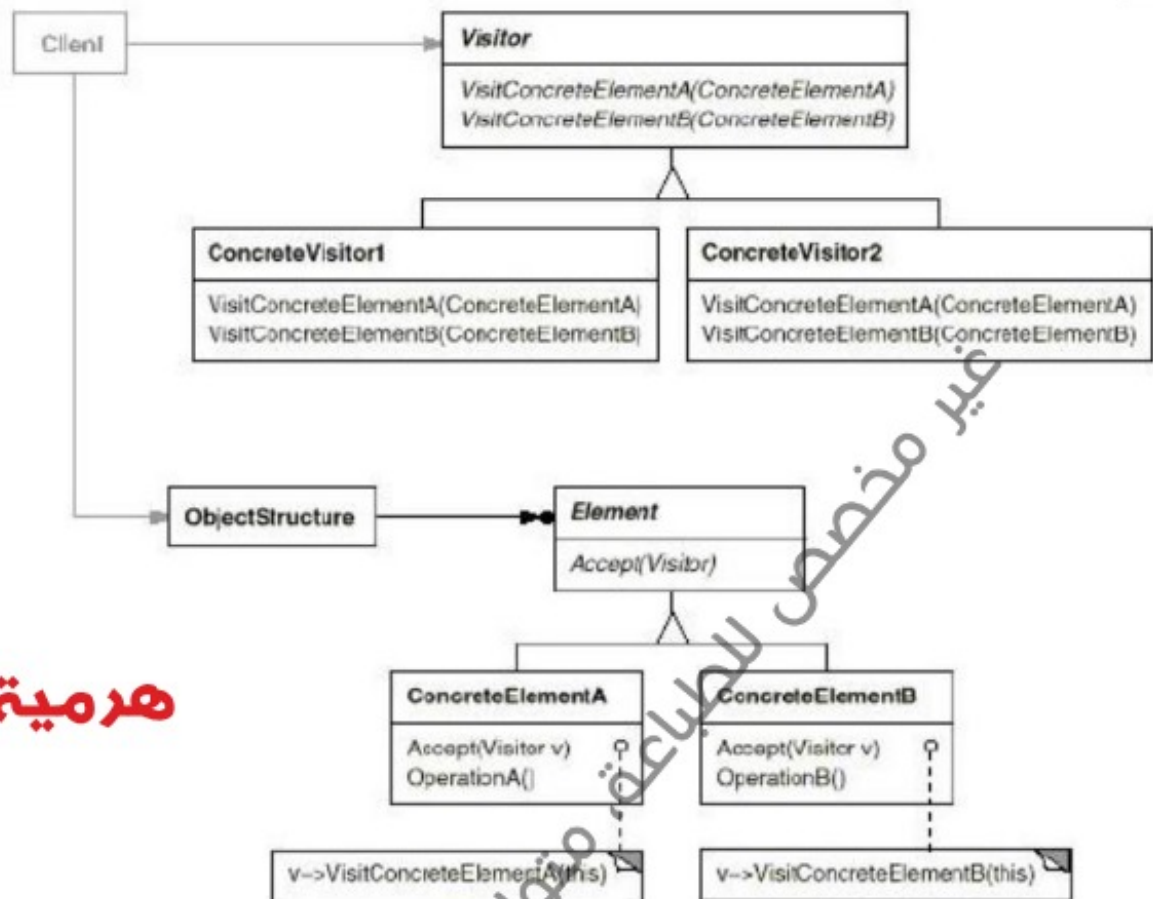
يسمح بتعريف عملية جديدة دون أن نغير في بنية الكلاس.

جرت العادة ان نقوم بعمل هرمية الصفوف التي تحوي الداتا تعرف العمليات على النظام داخلها، أما هذا التصميم يعتمد على هرميتين:

➤ هرمية البيانات.

➤ هرمية العمليات.

طالما هرمية البيانات ثابتة ممكن اضطر لعمليات جديدة، نقوم بتعريفها بهرمية ثانية وهي هرمية العمليات ، ونحتاج إلى طريقة لزيارة الداتا الموجودة والمحمولة بأغراض هرمية البيانات وعند التعديل نضيف عمليات جديدة بهرمية العمليات ، وبذلك أستطيع إضافة عمليات على هرمية المعطيات دون تعديلها.

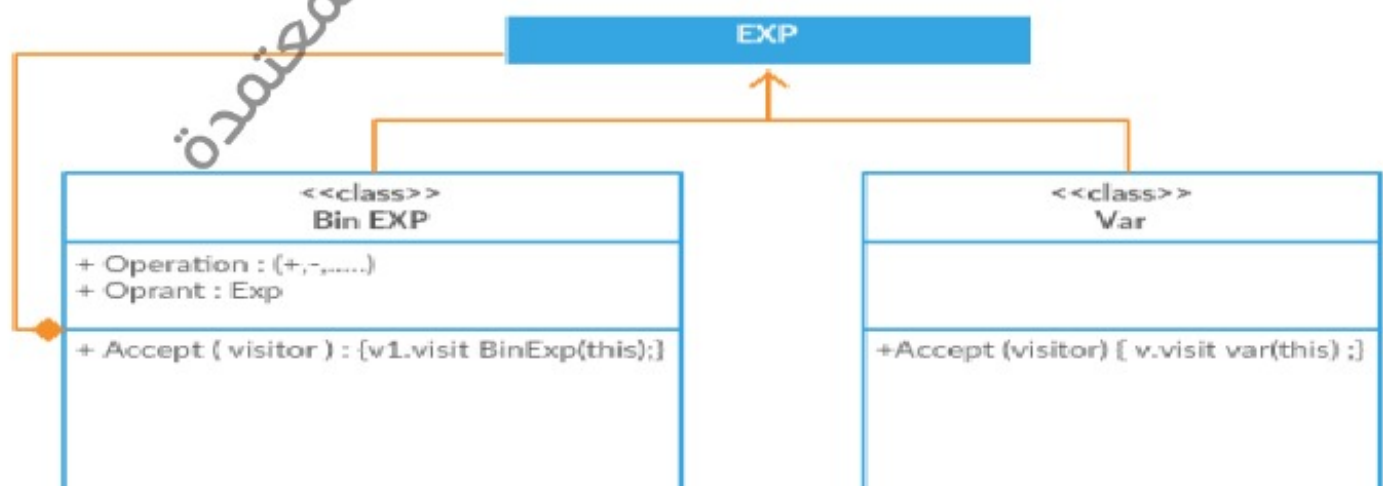


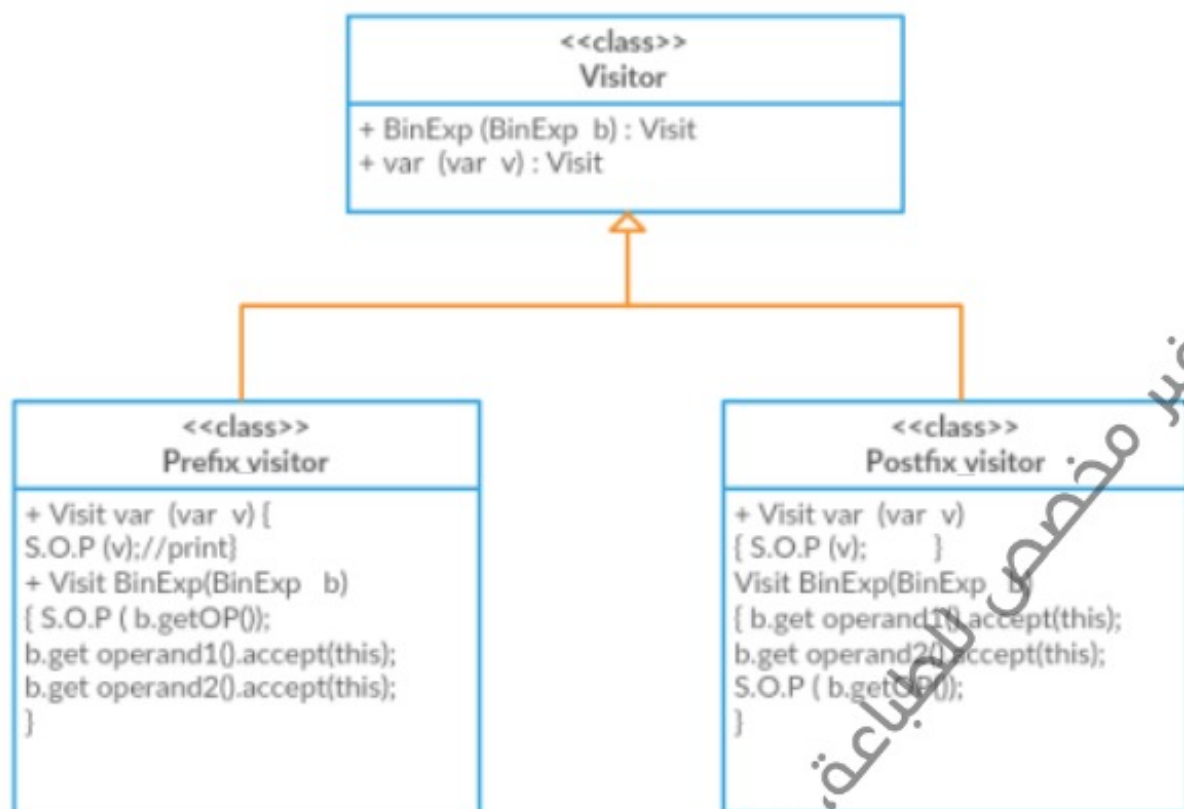
هرمية Visitor

ضمن صف ال **Element** نعرف عملية ال **accept** يمرر ال **visitor** كبارامتر ، ضمن الهرمية التالية عرفنا **interface** اسمها **visitor** نعرف ضمنها عملية **visit** يكون عددها بعدد الصفوف الحقيقية بالبنية الهرمية، فكل عملية نريد القيام بها على هذه العناصر نرث من ال **interface** ونقوم بعمل صف لتنفيذ هذين ال **method** بالطريقة المناسبة لزيارة البيانات.

ال **accept** التي تم إضافتها ببنية المعلومات هي نقطة الدخول.

مثال: بناء **expression** معين:





على فرض كان expression كالتالي: $5 + 2 * 3$

- طريقة تنفيذ prefix:

```

BinExp e1 = new BinExp(*, 2, +);
BinExp e = new BinExp(+, 5, e1);
e.accept(new Prefix_visitor)
    
```

الخرج: $5 + 2 * 3$



انتهت المحاضرة